

Uso de Inteligencia Artificial durante el desarrollo

Entregable del punto 10 de las bases. Describe qué herramienta se usó, en qué partes, qué se revisó y corrigió a mano, y qué decisiones tomó el equipo y no la IA.

Herramienta

Claude Code (Anthropic), ejecutado desde la terminal sobre el repositorio del proyecto. Es un agente que lee y escribe archivos, corre comandos y ejecuta las pruebas, así que cada cambio se verificó ejecutándolo, no solo leyéndolo.

Se usó dentro de la aplicación de escritorio de Claude, que integra terminal, navegador y control de versiones en la misma sesión, así que la IA pudo editar, ejecutar, abrir la aplicación y hacer commit sin cambiar de herramienta.

Servidores MCP utilizados

Además de los archivos y la terminal, el agente se conectó a estos servidores MCP (Model Context Protocol) para verificar el resultado en un navegador real:

Servidor MCP	Para qué se usó
Claude in Chrome	Recorrer el panel de administración desplegado con la sesión del equipo ya iniciada: escritorio, granjas, generaciones, alertas y formularios. La IA nunca escribió la contraseña; usó la sesión abierta en el navegador del equipo
Navegador integrado (Claude Browser)	Emular un viewport de 375 × 812 sobre la URL de producción, abrir el menú móvil, medir <code>scrollWidth</code> contra <code>clientWidth</code> y leer la consola en busca de errores. Así se detectó el desborde de las cifras del tablero

Ambos permitieron que la verificación fuera sobre la aplicación desplegada y no sobre una suposición.

Prompts

Los prompts de cada fase de construcción están en [PLAN.md](#) (secciones "FASE 1" a "FASE 7") y los de la conversación, en orden cronológico y con su contexto, en [PROMPTS.md](#).

Cómo se dirigió el trabajo

El proyecto lo desarrolló una sola persona (Nehemías Pérez Palma, carné 0909-26-9887), que actuó como director del trabajo de la IA: definió qué construir, revisó cada resultado y tomó las decisiones listadas más abajo.

Antes de escribir código se pidieron a la IA, a partir del PDF del reto, tres documentos que luego el equipo revisó y que la IA estaba obligada a respetar en todo el desarrollo:

Documento	Qué fija
CLAUDE.md	Reglas permanentes: dominio en español, nada de columnas para valores derivados, constantes desde <code>config/solar.php</code> , ejecutar el código antes de darlo por terminado
docs/PLAN.md	Esquema de base de datos cerrado, fases de trabajo y criterios de aceptación
docs/DISEÑO.md	Sistema de diseño vinculante: paleta, tipografía, densidad, prohibiciones

Sin esos documentos, un agente produce el resultado promedio de internet: tarjetas con sombra, gradientes violetas, Inter y los colores por defecto de Chart.js. Con ellos, produce lo que el equipo decidió.

El trabajo avanzó por fases, cada una con un objetivo único y su commit: andamiaje, modelo de datos, CRUDs, alertas, estadísticas, frontend público, API, datos de demostración y documentación.

Dónde se usó

Fase	Aporte de la IA	Qué revisó y corrigió el equipo
Andamiaje	Instalación de PHP 8.4, Composer, Laravel 13, Filament 5 y Scramble	La instalación inicial de PHP no traía <code>intl</code> ni <code>pdo_pgsql</code> ; se detectó al revisar <code>php -m</code> y se reinstaló desde la distribución oficial con un <code>php.ini</code> propio
Modelo de datos	Migraciones, modelos, enums, casts y relaciones a partir del esquema ya definido	El esquema es decisión del equipo. Se verificó en tinker que los 22 departamentos cargan y que las coordenadas caen dentro de Guatemala
CRUDs Filament	Recursos, formularios, tablas y el relation manager de paneles	Las firmas de Filament 5 se verificaron leyendo <code>vendor/filament</code> y los archivos que generan los comandos <code>make: </code> , no de memoria. Se corrigieron etiquetas, slugs y el formato numérico, que salía con punto de miles por el locale español
Alertas	Observador, servicio, comando artisan y recurso de solo lectura	Se definió que el 80% exacto sí dispara alerta y se escribieron pruebas para 79%, 80% y 81%
Proyección	Implementación de mínimos cuadrados y de la comparación contra datos reales	El método lo eligió el equipo. Se agregaron el piso en cero, el cambio de método con pocos períodos y las pruebas con serie lineal y serie plana
Frontend público	Blade, Tailwind, Leaflet y Chart.js siguiendo <code>docs/DISEÑO.md</code>	Se probó a 375 px, se revisó que no hubiera desbordamiento horizontal y se ajustó el encabezado en móvil
API REST	Controladores, API Resources, validación y anotaciones para Scramble	Se encontraron y corrigieron tres fallos reales al probar los endpoints: faltaba definir el limitador <code>api</code> , el parámetro <code>meses</code> reventaba si no venía, y el filtro <code>activa=true</code> no pasaba la validación booleana
Datos de demostración	Seeder con estacionalidad, tendencias y granjas en bajo desempeño	Los parámetros los fijó el equipo: 35 granjas, 18 meses, 6 granjas con desempeño bajo sostenido, estacionalidad de verano seco

Fallos de la IA que hubo que corregir

Se listan porque demuestran que el código generado se revisó en lugar de aceptarse:

1. **Tiles de mapa obsoletos.** El diseño indicaba CARTO Positron "sin API key". Al probarlo, CARTO devolvía un mosaico gris con el texto "API KEY REQUIRED". Se descargó el mosaico para confirmarlo y se cambió a la base gris de Esri, que cumple el mismo criterio de mapa monocromo sin clave.
2. **Limitador de peticiones inexistente.** Las rutas usaban `throttle:api`, que en Laravel 13 ya no viene definido por omisión. Los diez endpoints respondían 500 hasta que se definió el limitador.
3. **Parámetro opcional mal leído.** `$request->validate(...)[ 'meses' ]` lanzaba excepción cuando el parámetro no venía. Solo se detectó porque se probó el endpoint sin argumentos.
4. **Orden de la relación en la proyección.** La relación `generaciones` ordena ascendente, así que `orderByDesc` no reemplazaba el orden y la regresión tomaba los 12 períodos más antiguos en lugar de los más recientes. Se corrigió con `reorder`.
5. **Widgets en carga diferida.** El escritorio quedaba en "Cargando..." indefinidamente. Se desactivó la carga diferida de los widgets y del relation manager.
6. **Formato numérico.** Con el locale en español, Filament mostraba `414.432` en lugar de `414,432`. Se forzó el locale numérico a inglés solo para las cifras, conservando las fechas en español.
7. **Palabra reservada en MySQL.** La serie nacional usaba `real` como alias de columna. SQLite lo acepta; MySQL en producción respondía 500. Se detectó al abrir la URL desplegada y se corrigió renombrando los alias.
8. **Acceso al panel en producción.** El login de Filament funcionaba en local pero en producción devolvía 403 tras autenticarse. Filament solo deja pasar a cualquier usuario en entorno `local`; fuera de él exige que el modelo `User` implemente `FilamentUser::canAccessPanel()`. La IA no lo había previsto; se detectó al probar el panel desplegado.
9. **Cifras desbordadas en móvil.** En 375 px, `260,191` y `308.13 GWh` se salían de su celda en el tablero. Se detectó midiendo `scrollWidth` contra `clientWidth` en producción y se corrigió con `clamp()` en el tamaño de fuente.

Qué decidió el equipo y no la IA

- **El esquema de base de datos**, incluida la decisión de que capacidad instalada, generación acumulada y CO₂ evitado sean valores calculados y no columnas.
- **El stack:** Laravel con Filament para el panel interno y Blade propio para el frontend público, en vez de dejar todo dentro de Filament.
- **El método de proyección** y sus salvaguardas.
- **La regla del umbral** en el caso borde del 80% exacto.
- **El sistema de diseño completo:** una sola escala de color para generación en todas las vistas, sin sombras, sin gradientes, con cifras tabulares y densidad de consola de operación.
- **Los parámetros de los datos de demostración**, para que el tablero, el mapa, las alertas y las proyecciones cuenten una historia coherente.

Verificación

Todo el código generado se ejecutó antes de darlo por bueno:

- `php artisan migrate:fresh --seed` corre limpio y reporta 8 modelos, 35 granjas y 19 alertas.
- `php artisan test` pasa 17 pruebas con 71 aserciones.
- Los trece endpoints de la API se consultaron uno por uno verificando código de estado y forma de la respuesta, incluidos los casos 404 y 422.
- Las cinco pantallas públicas y el panel se recorrieron en el navegador, incluido crear una generación desde el formulario y comprobar que la alerta aparece sola.